

IES El Caminàs	CFGS Administración de Sistemas Informáticos en Red
Módulo: Administración de Sistemas Operativos	Curso: 2º ASIR
Nombre y apellidos:	

Práctica: scripting en Bash con expresiones regulares extendidas

Introducción

La práctica consiste en realizar cinco scripts que usan expresiones regulares extendidas con el operador `==` de Bash. Además de los operadores básicos (`^` `$` `.` `*` `[` `]` `[^` `]`), ahora se pueden utilizar los extendidos:

Operador	Significado	Ejemplo
<code>+</code>	El elemento anterior 1 o más veces	<code>^[0-9]+\$</code>
<code>?</code>	El elemento anterior 0 o 1 vez	<code>^https?://</code>
<code>{n}</code>	Exactamente n veces	<code>^[0-9]{8}\$</code>
<code>{n,m}</code>	Entre n y m veces	<code>^[a-z]{3,16}\$</code>
<code>a b</code>	a o b	<code>\.conf\$ \.cfg\$</code>
<code>()</code>	Agrupar elementos	<code>^(in)?útil\$</code>

Como en la práctica anterior, la expresión se guarda en una variable y se escribe sin comillas después de `==`:

```
re='^[0-9]{3,5}$'
if [[ $puerto == $re ]]; then
    echo "Tiene entre 3 y 5 dígitos"
fi
```

Entrega

Los cinco scripts deben subirse al repositorio de ASO, dentro de la carpeta `bash/sesion4_regex_extendidas`:

```
bash/
├── sesion4_regex_extendidas/
│   ├── validar_ip.sh
│   ├── validar_mac.sh
│   ├── correo_centro.sh
│   ├── comprobar_password.sh
│   └── tipo_ip.sh
```

Ejercicio 1. Formato de una dirección IP

Escribe `validar_ip.sh`, que recibe una dirección IPv4 como parámetro e indica si tiene el formato correcto: cuatro números de 1 a 3 cifras separados por puntos.

- Si no recibe ningún parámetro, muestra un mensaje de uso y termina.
- La comprobación se hace con una sola expresión regular.

```
$ ./validar_ip.sh 192.168.10.1
192.168.10.1 tiene formato de IP
$ ./validar_ip.sh 192.168.1
192.168.1 no tiene formato de IP
$ ./validar_ip.sh 1.2.3.4.5
1.2.3.4.5 no tiene formato de IP
$ ./validar_ip.sh 192.168.1.a
192.168.1.a no tiene formato de IP
```

Pregunta: ¿qué responde tu script con 999.999.999.999? ¿Es una IP válida? Responde en un comentario al final del script.

Ejercicio 2. Validar una dirección MAC

Escribe `validar_mac.sh`, que recibe una dirección MAC como parámetro e indica si es válida. Una MAC válida tiene seis pares de dígitos hexadecimales (del 0 al 9 y de la a a la f, en mayúsculas o minúsculas) separados por : o por -.

La comprobación se hace con una sola expresión regular.

```
$ ./validar_mac.sh 08:00:27:3a:1f:01
08:00:27:3a:1f:01 es una MAC válida
$ ./validar_mac.sh 08-00-27-3A-1F-02
08-00-27-3A-1F-02 es una MAC válida
$ ./validar_mac.sh 08:00:27:3a:1f:4
08:00:27:3a:1f:4 no es una MAC válida
$ ./validar_mac.sh 08:00:27:3a:1f:0g
08:00:27:3a:1f:0g no es una MAC válida
```

Ejercicio 3. Cuentas de correo del centro

Escribe `correo_centro.sh`, que recibe una dirección de correo como parámetro e indica si es una cuenta educativa de Conselleria y de qué tipo:

- Profesorado: `nombre@edu.gva.es`
- Alumnado: `nombre@alu.edu.gva.es`

El nombre tiene al menos un carácter y solo puede contener letras minúsculas, dígitos, puntos, guiones o guiones bajos.

- Si no recibe ningún parámetro, muestra un mensaje de uso y termina.
- Comprueba con una sola expresión regular que la dirección es de alguno de los dos tipos.
- Si lo es, indica si pertenece a profesorado o a alumnado.

```
$ ./correo_centro.sh jgarcia@edu.gva.es
jgarcia@edu.gva.es es una cuenta de profesorado
$ ./correo_centro.sh lucia.ferrer@alu.edu.gva.es
lucia.ferrer@alu.edu.gva.es es una cuenta de alumnado
$ ./correo_centro.sh nerea@alu.edu.gva
nerea@alu.edu.gva no es una cuenta educativa
$ ./correo_centro.sh pepe@gmail.com
pepe@gmail.com no es una cuenta educativa
```

Ejercicio 4. Política de contraseñas

Escribe `comprobar_password.sh`, que recibe una contraseña como parámetro y comprueba si cumple la política de contraseñas del centro:

1. Tiene al menos 12 caracteres.
2. Contiene algún dígito.
3. Contiene alguna letra mayúscula.
4. Contiene algún símbolo, es decir, algún carácter que no sea una letra ni un dígito.
5. No es una palabra del diccionario `/usr/share/dict/spanish`, sin distinguir mayúsculas de minúsculas.
6. Si no recibe ningún parámetro, muestra un mensaje de uso y termina.
7. Las condiciones 1 a 4 se comprueban con una expresión regular cada una. La condición 5, con `grep`.
8. El script muestra el resultado de todas las condiciones, no solo de la primera que falle, y al final indica si la contraseña es segura.

```
$ ./comprobar_password.sh administrador
Tiene al menos 12 caracteres: sí
Contiene algún dígito: no
Contiene alguna mayúscula: no
Contiene algún símbolo: no
No es una palabra del diccionario: no
La contraseña no es segura

$ ./comprobar_password.sh 'Caminas2026!'
Tiene al menos 12 caracteres: sí
Contiene algún dígito: sí
Contiene alguna mayúscula: sí
Contiene algún símbolo: sí
No es una palabra del diccionario: sí
La contraseña es segura
```

Si la contraseña contiene símbolos, hay que escribirla entre comillas simples al ejecutar el script. Si no, la shell puede interpretar algunos de ellos.

Pregunta: `Caminas2026!` cumple todas las condiciones. ¿Te parece realmente una contraseña segura? ¿Por qué? Responde en un comentario al final del script.

Ejercicio 5. Tipo de dirección IP

Escribe `tipo_ip.sh`, que recibe una dirección IP como parámetro e indica de qué tipo es:

- Loopback: empieza por `127.`
- Privada: empieza por `10.`, por `192.168.` o por `172.` seguido de un número del 16 al 31.
- Pública: cualquier otra.

La comprobación de si es privada se hace con una sola expresión regular.

```
$ ./tipo_ip.sh 127.0.0.1
127.0.0.1 es una dirección loopback
$ ./tipo_ip.sh 172.20.0.5
172.20.0.5 es una dirección privada
$ ./tipo_ip.sh 172.32.0.1
172.32.0.1 es una dirección pública
$ ./tipo_ip.sh 8.8.8.8
8.8.8.8 es una dirección pública
```

Pregunta: el rango de 172.16 a 172.31 no se puede escribir con una sola clase. ¿Cómo lo has resuelto?
Responde en un comentario al final del script.